KUBERNETES

FIELD CHEAT SHEET

For DevOps & Platform Engineers — Debug, Operate, Ship

Covers: kubectl | Debugging | Networking | RBAC | Storage | Helm | Performance

by Pushkaraj Sant • DevSecOps Engineer • Kubernetes Security Specialist

01 CLUSTER & CONTEXT COMMANDS

Command	What It Does
`kubectl version --short`	Client + Server version in one line
`kubectl cluster-info`	API server, CoreDNS, dashboard endpoints
`kubectl get nodes -o wide`	All nodes with IPs, OS, kernel, runtime
`kubectl top nodes`	CPU/Memory usage per node (needs metrics-server)
`kubectl config get-contexts`	List all configured contexts
`kubectl config use-context <ctx>`	Switch active context
`kubectl config current-context`	Show which context is active right now
`kubectl config view --minify`	Show only the active context config
`kubectl api-resources`	All available resource types + shortnames
`kubectl api-versions`	All API versions the server supports
`kubectl explain pod.spec.containers`	Built-in docs for any field
`kubectl get componentstatuses`	Scheduler, controller, etcd health

02 NAMESPACES, LABELS & SELECTORS

▸ Namespace Operations

Command	Purpose
`kubectl get ns`	List all namespaces
`kubectl create ns <name>`	Create namespace quickly
`kubectl delete ns <name>`	Delete ns + ALL resources inside it
`kubectl get all -n <ns>`	Every resource in a namespace
`kubectl get all --all-namespaces`	Every resource cluster-wide
`-n kube-system`	Flag to scope command to kube-system

▸ Label & Selector Filters

Command	Purpose
`kubectl get pods -l app=nginx`	Filter by label
`kubectl get pods -l 'env in (prod,staging)'`	Filter with set-based selector
`kubectl get pods -l env!=dev`	Exclude label value
`kubectl label pod <pod> tier=frontend`	Add/update label on a pod
`kubectl label pod <pod> tier-`	Remove a label from a pod

Command	Purpose
`kubectl annotate pod <pod> owner=team-a`	Add annotation
`kubectl get pods --show-labels`	Show all labels in output
`kubectl get pods --field-selector status.phase=Running`	Filter by field selector

03 POD DEBUGGING (Most Used in Real Incidents)

▸ Pod Status & Inspection

Command	What to Look For
`kubectl get pods -o wide`	Node placement, IP, restarts, age
`kubectl describe pod <pod>`	Events section — image pull errors, OOM, mounts
`kubectl get pod <pod> -o yaml`	Full spec: resources, probes, env, volumes
`kubectl get pod <pod> -o jsonpath='{.status.phase}'`	Just the status field
`kubectl get events --sort-by=.metadata.creationTimestamp`	Timeline of all events
`kubectl get events -n <ns> --field-selector involvedObject.name=<pod>`	Events for a specific pod

▸ Logs

Command	Use Case
`kubectl logs <pod>`	Current container logs
`kubectl logs <pod> -c <container>`	Multi-container pod — pick one
`kubectl logs <pod> --previous`	Logs from the CRASHED previous instance
`kubectl logs <pod> -f`	Follow/stream logs live
`kubectl logs <pod> --tail=100`	Last 100 lines only
`kubectl logs <pod> --since=1h`	Logs from last 1 hour
`kubectl logs -l app=nginx --all-containers`	Logs from ALL pods with label
`kubectl logs <pod> 2>&1 | grep -i error`	Grep errors inline

▸ Exec & Debug Containers

Command	Use Case
`kubectl exec -it <pod> -- /bin/sh`	Shell into a running container
`kubectl exec -it <pod> -c <c> -- bash`	Shell into specific container
`kubectl exec <pod> -- env`	Dump environment variables
`kubectl exec <pod> -- cat /etc/hosts`	Inspect hosts file inside pod
`kubectl debug -it <pod> --image=busybox --copy-to=debug-pod`	Clone pod with debug image
`kubectl debug node/<node> -it --image=ubuntu`	Debug directly on a node
`kubectl run tmp --rm -it --image=busybox --restart=Never -- sh`	Disposable debug pod, auto-deletes

Command	Use Case
`kubectl run curl-test --rm -it --image=curlimages/curl -- sh`	Test HTTP from inside cluster

⚠ *CrashLoopBackOff? Always check --previous logs first, then describe for OOMKilled events.*

✔ *ImagePullBackOff = wrong image name OR missing imagePullSecret for private registry.*

04 DEPLOYMENTS, REPLICASETS & DAEMONSETS

▸ Deployments

Command	Purpose
`kubectl get deploy -o wide`	All deployments with images
`kubectl describe deploy <name>`	Replicas, strategy, conditions, events
`kubectl rollout status deploy/<name>`	Watch rollout progress in real-time
`kubectl rollout history deploy/<name>`	All revision history
`kubectl rollout undo deploy/<name>`	Instant rollback to previous revision
`kubectl rollout undo deploy/<name> --to-revision=3`	Rollback to a specific revision
`kubectl scale deploy <name> --replicas=5`	Manually scale replicas
`kubectl set image deploy/<name> app=nginx:1.25`	Update container image
`kubectl annotate deploy <name> kubernetes.io/change-cause='v2 release'`	Record reason in history
`kubectl autoscale deploy <name> --min=2 --max=10 --cpu-percent=70`	Create HPA instantly

▸ DaemonSets & StatefulSets

Command	Purpose
`kubectl get ds -A`	All DaemonSets cluster-wide
`kubectl describe ds <name> -n <ns>`	Node selector, update strategy, events
`kubectl rollout status ds/<name>`	DaemonSet rollout status
`kubectl rollout restart ds/<name>`	Rolling restart of DaemonSet
`kubectl get sts -o wide`	All StatefulSets
`kubectl rollout status sts/<name>`	StatefulSet rollout (respects ordering)

▸ Jobs & CronJobs

Command	Purpose
`kubectl get jobs -A`	List all jobs
`kubectl describe job <name>`	Completions, active, failed counts
`kubectl logs job/<name>`	Logs from job pod
`kubectl create job debug --from=cronjob/<name>`	Manually trigger a CronJob now
`kubectl get cronjob`	List scheduled jobs with last schedule time

05 SERVICES, NETWORKING & INGRESS DEBUGGING

▸ Service Inspection

Command	Purpose
`kubectl get svc -o wide`	Services with selectors, external IPs, ports
`kubectl describe svc <name>`	Endpoints, ports, events — check selector match
`kubectl get endpoints <name>`	IP:Port of pods behind the service
`kubectl get ep -A | grep '<none>'`	Services with NO backing pods — mismatch!
`kubectl port-forward svc/<name> 8080:80`	Local tunnel to service port
`kubectl port-forward pod/<pod> 8080:80`	Local tunnel directly to pod

▸ DNS Debugging

Command	Purpose
`kubectl run dnsutils --image=registry.k8s.io/e2e-test-images/jessie-dnsutils:1.3 --rm -it --nslookup kubernetes`	Official DNS test pod
`kubectl exec -it <pod> -- nslookup <svc>.<ns>.svc.cluster.local`	Resolve service DNS from inside cluster
`kubectl exec -it <pod> -- cat /etc/resolv.conf`	Check cluster DNS config inside pod
`kubectl get pods -n kube-system -l k8s-app=kube-dns`	CoreDNS pod health
`kubectl logs -n kube-system -l k8s-app=kube-dns`	CoreDNS logs for resolution failures

▸ Ingress

Command	Purpose
`kubectl get ingress -A`	All ingress rules cluster-wide
`kubectl describe ingress <name>`	Rules, backend, TLS, events
`kubectl get ingressclass`	Available ingress controllers
`kubectl logs -n <ns> deploy/ingress-nginx-controller`	NGINX ingress controller logs

⚠ *Endpoint = <none>? Service selector does NOT match pod labels. This is the #1 networking mistake.*

✔ *Test pod-to-pod connectivity: kubectl exec from pod A, curl pod-B-IP:port directly.*

06 STORAGE: PV, PVC & STORAGECLASSES

Command	Purpose
`kubectl get pv`	All Persistent Volumes + capacity, status, reclaim
`kubectl get pvc -A`	All PVCs — check Bound vs Pending
`kubectl describe pvc <name>`	Events: provisioning errors, access mode mismatch
`kubectl get sc`	StorageClasses available + provisioner
`kubectl get pv -o custom-columns=NAME:.metadata.name,CLAIM:.spec.claimRef.name,STATUS:.status.phase`	Clean PV status view
`kubectl describe pv <name>`	Which PVC it's bound to, access modes, mount options
`kubectl patch pv <name> -p '{"spec":{"persistentVolumeReclaimPolicy":"Retain"}}'`	Protect PV from deletion
`kubectl get volumeattachments`	CSI volume attachment status per node

⚠ *PVC stuck in Pending? Check: StorageClass exists, correct access mode, provisioner is running.*

✔ *PV in Released state? Edit PV and remove spec.claimRef to make it Available again.*

07 CONFIGMAPS & SECRETS

Command	Purpose
`kubectl get cm -n <ns>`	List ConfigMaps in namespace
`kubectl describe cm <name>`	View all key-value data
`kubectl get cm <name> -o jsonpath='{.data}'`	Extract just the data section
`kubectl create cm app-config --from-file=./config.properties`	Create CM from a file
`kubectl create cm app-config --from-literal=key=value`	Create CM from literal
`kubectl get secret -n <ns>`	List secrets (values are base64 hidden)
`kubectl get secret <name> -o jsonpath='{.data.password}' | base64 -d`	Decode a secret value
`kubectl create secret generic db-creds --from-literal=pass=secret123`	Create secret from literal
`kubectl create secret docker-registry regcred --docker-server=... --docker-username=... --docker-password=...`	Create image pull secret
`kubectl get secret regcred -o jsonpath='{.data.\.dockerconfigjson}' | base64 -d | jq`	Inspect pull secret content

⚠ *Never store Secrets in Git. Use Sealed Secrets, External Secrets Operator, or Vault.*

08 NODE DEBUGGING & MAINTENANCE

▸ Node Health

Command	Purpose
`kubectl get nodes`	Node status — Ready vs NotReady
`kubectl describe node <node>`	Conditions, capacity, allocatable, taints, events
`kubectl top node <node>`	CPU/Memory live usage
`kubectl get node <node> -o jsonpath='{.status.conditions}'`	All conditions as JSON
`kubectl get node -o custom-columns=NAME:.metadata.name,READY:.status.conditions[?(@.type=='Ready')].status`	Ready status column only

▸ Taints, Draining & Cordon

Command	Purpose
`kubectl cordon <node>`	Mark node unschedulable — no new pods

Command	Purpose
`kubectl uncordon <node>`	Re-enable scheduling on node
`kubectl drain <node> --ignore-daemonsets --delete-emptydir-data`	Safely evict all pods before maintenance
`kubectl taint node <node> key=value:NoSchedule`	Add taint to node
`kubectl taint node <node> key:NoSchedule-`	Remove taint (trailing dash = delete)
`kubectl get nodes -o custom-columns=NAME:.metadata.name,TAINTS:.spec.taints`	View all node taints

▸ Pod Affinity & Scheduling Debugs

Command	Purpose
`kubectl get pods -o wide | grep Pending`	Find unscheduled pods
`kubectl describe pod <pending-pod>`	Look at Events: 0/3 nodes available — why
`kubectl get pods --field-selector=spec.nodeName=<node>`	All pods on a specific node

09 RBAC DEBUGGING

Command	Purpose
`kubectl auth can-i get pods --as=system:serviceaccount:ns:sa`	Check if SA has permission
`kubectl auth can-i '*' '*' --as=admin`	Check if user has all permissions
`kubectl get rolebinding,clusterrolebinding -A | grep <subject>`	Find all bindings for a user/SA
`kubectl describe clusterrolebinding <name>`	Who is bound + what role
`kubectl get clusterrole <name> -o yaml`	See exact rules (verbs, resources, apiGroups)
`kubectl get serviceaccount -n <ns>`	List SAs in namespace
`kubectl create token <sa-name> -n <ns>`	Generate SA token on demand
`kubectl auth reconcile -f rbac.yaml`	Apply RBAC safely (idempotent)

⚠ *403 Forbidden? Use 'kubectl auth can-i' to pinpoint exact missing verb+resource.*

✔ *Always scope RoleBindings to namespaces. Use ClusterRoleBinding only when truly needed.*

10 RESOURCE QUOTAS, LIMITS & HPA

Command	Purpose
`kubectl get resourcequota -n <ns>`	Quota limits + current usage per namespace
`kubectl describe resourcequota -n <ns>`	Detailed breakdown of each resource
`kubectl get limitrange -n <ns>`	Default requests/limits applied to pods
`kubectl get hpa -A`	All HPAs: min/max replicas, current target
`kubectl describe hpa <name>`	Current metrics vs target — why it scaled
`kubectl top pods -n <ns> --sort-by=memory`	Top memory consumers
`kubectl top pods -A --sort-by=cpu`	Top CPU consumers cluster-wide
`kubectl get pods -o json | jq '.items[].spec.containers[].resources'`	All resource requests/limits as JSON

⚠ *OOMKilled? Container hit memory limit. Increase limit or find the memory leak.*

✔ *Always set requests AND limits. Pods without requests affect cluster scheduling accuracy.*

11 HELM — DEPLOY, DEBUG & ROLLBACK

Command	Purpose
`helm list -A`	All releases across all namespaces
`helm status <release>`	Release status + notes
`helm get values <release>`	Values used in deployed release
`helm get manifest <release>`	All rendered K8s manifests from the release
`helm history <release>`	Full revision history
`helm rollback <release> <revision>`	Rollback to a specific revision
`helm upgrade <rel> <chart> --values=vals.yaml --dry-run`	Test upgrade without applying
`helm template <rel> <chart> --values=vals.yaml`	Render templates locally without installing
`helm lint <chart-dir>`	Validate chart for errors before install
`helm uninstall <release> --keep-history`	Remove but keep history for auditing
`helm plugin install https://github.com/databus23/helm-diff`	Install helm-diff plugin
`helm diff upgrade <rel> <chart>`	Show diff before upgrading (requires plugin)

12 POWER ONE-LINERS EVERY DEVOPS ENGINEER NEEDS

One-Liner	What It Does
`kubectl get pods -A | grep -v Running | grep -v Completed`	Show only unhealthy pods cluster-wide
`kubectl get pods -A -o wide | grep <node-name>`	All pods running on a specific node
`kubectl delete pod <pod> --grace-period=0 --force`	Force delete a stuck terminating pod
`kubectl rollout restart deploy -n <ns>`	Rolling restart ALL deployments in namespace
`kubectl get pods --field-selector=status.phase=Failed -A`	Find all failed pods
`kubectl get pods -A --sort-by=.metadata.creationTimestamp | tail`	Newest pods at the bottom
`for ns in $(kubectl get ns -o name); do echo $ns; kubectl get pods -n ${ns##*/} | grep -v Running; done`	Scan all namespaces for non-running pods
`kubectl get nodes -o jsonpath='{.items[*].status.addresses[?(@.type=="InternalIP")].address}'`	List all node IPs
`kubectl get pods -o json | jq '.items[] | select(.status.containerStatuses[].restartCount > 5) | .metadata.name'`	Pods with >5 restarts

One-Liner	What It Does
`kubectl diff -f manifest.yaml`	Diff what would change if you apply this manifest
`kubectl apply --dry-run=server -f manifest.yaml`	Server-side dry-run — catches more errors
`kubectl neat get pod <pod> -o yaml`	Clean YAML without managed fields (needs kubectl-neat plugin)
`watch kubectl get pods -n <ns>`	Live-watching pod status every 2 seconds
`kubectl get events -A --sort-by='.lastTimestamp' | tail -20`	Last 20 events cluster-wide

13 COMMON ERROR PATTERNS & ROOT CAUSES

Error / Symptom	Likely Cause	Debug Command
`CrashLoopBackOff`	App crash, bad config, missing env var	`kubectl logs <pod> --previous`
`OOMKilled`	Memory limit exceeded	`kubectl describe pod — check lastState`
`ImagePullBackOff`	Wrong image, no pull secret, rate limit	`kubectl describe pod — Events section`
`Pending (scheduling)`	Insufficient resources, taints, affinity	`kubectl describe pod — 0/N nodes available`
`Pending (PVC)`	No StorageClass, wrong access mode	`kubectl describe pvc — Events section`
`CreateContainerError`	Bad command, missing volume/secret/CM	`kubectl describe pod — Events`
`Endpoints <none>`	Label selector mismatch on Service	`kubectl get ep + check pod labels`
`403 Forbidden (API)`	RBAC — SA missing verb/resource	`kubectl auth can-i get pods --as=...`
`Evicted`	Node out of disk or memory (eviction)	`kubectl describe pod — message field`
`Init:Error`	InitContainer failing — dependency not up	`kubectl logs <pod> -c <init-container>`
`Terminating stuck`	Finalizer blocking deletion	`kubectl patch pod <p> -p '{"metadata":{"finalizers":[]}}'`
`Node NotReady`	kubelet stopped, disk pressure, network	`kubectl describe node — Conditions`

14 SECURITY & POLICY DEBUGGING

Command	Purpose
`kubectl get psp (deprecated)`	PodSecurityPolicies — replaced by Pod Security Admission
`kubectl get ns <name> -o yaml | grep pod-security`	Check Pod Security Admission labels on NS
`kubectl label ns <ns> pod-security.kubernetes.io/enforce=restricted`	Enforce restricted policy on namespace
`kubectl get netpol -A`	All NetworkPolicies cluster-wide
`kubectl describe netpol <name>`	Ingress/egress rules, pod selectors
`kubectl get cm -n kube-system kube-apiserver-admission-control-config-file -o yaml`	Check admission controller config
`kubectl get constrainttemplates (OPA/Gatekeeper)`	All OPA Gatekeeper constraint templates
`kubectl get constraints -A`	All active Gatekeeper constraints
`kubectl describe constraint <name>`	Violations list — which pods are non-compliant
`kubectl get clusterpolicies (Kyverno)`	All Kyverno policies
`kubectl get policyreport -A`	Kyverno policy scan results per namespace
`kubectl get events -A | grep FailedCreate`	Policy-blocked pod creation events
`kubectl get pod <pod> -o yaml | grep -i securityContext`	Inspect pod/container security context
`falco --list | grep syscall`	List Falco monitored syscalls (if Falco installed)

15 CONTROL PLANE & ETCD HEALTH

Command	Purpose
`kubectl get pods -n kube-system`	All control plane pods running?
`kubectl logs -n kube-system kube-apiserver-<node>`	API server logs
`kubectl logs -n kube-system kube-controller-manager-<node>`	Controller manager logs
`kubectl logs -n kube-system kube-scheduler-<node>`	Scheduler logs
`kubectl logs -n kube-system etcd-<node>`	etcd logs — raft, leader election
`ETCDCTL_API=3 etcdctl endpoint health --endpoints=<url>`	etcd cluster health
`ETCDCTL_API=3 etcdctl member list`	etcd members + leader
`kubectl get lease -n kube-node-lease`	Node heartbeat leases — detect ghost nodes
`kubectl get --raw /healthz`	API server liveness check

Command	Purpose
`kubectl get --raw /readyz`	API server readiness check
`kubectl get --raw /metrics | grep apiserver_request_total`	API server request counts

⚠ *Always backup etcd before major cluster changes: etcdctl snapshot save backup.db*

Save this. Share this. Debug faster. | Follow Pushkaraj Sant on LinkedIn for more Kubernetes & DevSecOps content

#Kubernetes #DevOps #DevSecOps #CloudNative #CNCF #SRE #Platform Engineering